괴델은 임의의 자연 대상에 수를 부여함으로써 상위 수학명제로, 그리고 수학명제로, 그리고 괴델수로 격하시키는 방법을 보였다. 튜링은 여기에서 착안하여 계산 가능한 자연대상을 튜링기계(괴델수)로 격하시키는 방법을 보였다. 과학기술의 발전은 점점 가속화되고 있는데, 그것은 저차원적인 자연 대상을 기계로 자동화시키면서 점점 더 고차원적인 자연 대상을 기계로 격하시켜 자동화시켜나가기 때문이다. 그런데 이 과학기술 발전에는 언젠가 특이점이 올 것이라고 폰 노이만, 빌 조이, 레이 커즈와일 등이 경고했다. 이 특이점이란 초지능의 탄생을 뜻한다. 초지능이란 인간의 지능을 초월하는 인공지능인데, 초지능의 탄생이 특이점이라 불리우는 이유는 그 시점이 오면 블랙홀이 들어갔을 때 더 이상 인간의 능력으로 도무지 돌이킬 수 없듯이, 다시는 그 이후의 세상을 과거의 세상으로 돌이킬 수 없기 때문이다.

나는 이 초지능의 탄생이 과학기술 발전을 자동화시키는 시점이라고 생각한다. 또 과학기술 발전의 자동화는 인간이 역사적으로 기술공학을 발전시켜 오면서 자연대상을 기계로 자동화해온 과정 자체를 기계로 격하시키는 것이라고 생각한다. 나는 이것이 인간이 지금까지 해오던 자연 대상을 상위 수학명제로, 수학명제로, 괴델수(튜링기계) 로 격하시키는 과정 자체를 상위 수학명제로, 수학명제로, 괴델수(튜링기계) 로 격하시키는 것으로 성취될 수 있다고 생각한다.

그러므로 자연대상을 상위 수학명제로, 수학명제로, 괴델수(튜링기계)로 격하시키는 패턴을 포착하기 위하여 이것에 관한 데이터가 필요하다.

1: 08/31

- 최대 실행시간은 거의 항상 1 초 이다.
- P1 나옴. 기한 내에 코드 제출 및 보고서 제출.

P1

!!! note ""

```
문제 : 접속 횟수를 카운팅하는 어떤 웹사이트에 대한 카운팅 로직이 잘못되어

$3$ 이 $4$ 로 증가하는 것이 아니라 $5$ 로 증가하는 상황에 대하여

웹사이트가 카운팅한 숫자를 올바르게 고쳐라.
```

자연 대상을 수학 명제로 사상시키기

이때 편의상 제한 조건도 함께 포함시켰다.

!!! note "명제 p"

```
$0 < n < 10 ^{8}$ 인 $n \in \N$ 을 $1$ 씩 증가시킬 때,

$n$ 을 구성하는 숫자 집합 $S$ 의 원소 $m \in S$ 이 $n$ 이 증가함에 따라 바뀌어야 하는데 $m = 3$ 에서 $m = 5$ 로 증가되었다면,

이 $n$ 을 일반적인 자연수 증가 방식에 따른 증가 결과값으로 변환할 수 있다.
```

명제 p 증명

!!! note "가설 1"

```
자연수 $n \in \N$ 을 $1$씩 증가시킬 때,

$n$ 을 구성하는 숫자 집합 $S$ 의 원소 $m \in S$ 가 항상 $m = 3$ 에서 $m = 5$ 로 바뀌었다면,

이 $n$ 과 일반적인 자연수 증가 방식에 따른 증가 결과값 $s$ 과의 차이를 계산할 수 있는 함수 $f(n)$ 이 존재한다.
```

가설 1 이 주장하는 함수 $f(n)$ 이 존재하면 $n - f(n)$ 로 일반적인 자연수 증가 방식에 따라 증가한 자연수 $s \in \N$ 을 구할 수 있으므로 명제 p 가 증명된다.

그런데 아래의 증명을 통하여 $f(n)$ 이 존재한다는 것이 귀납적으로(비도덕적인(?) 방식으로) 증명되었다. 그러므로 가설 1 이 증명되었다. 이에 따라 명제 p 도 증명되었고, 따라서 명제 p 는 계산가능하다.■

가설 1 증명

자연수 $n \in \N$ 은 $i \in \N$ 에 대한 $a_i \in {0,1,2,3,5,6,7,8,9}$ 에 대하여 $n = \dots a_i \dots a_5a_4a_3a_2a_1$ 로 구성된다.

패턴 관찰과 1차 추상화

이때 n 을 일반적인 자연수 증가 방식에 따른 증가 결과값으로 변환한 자연수를 $s \in \N$ 이라고 하자.

n	s
3	3
5	4

위 표에서 나타난 자연수 증가 패턴을 귀납적으로 추상화시켜 다음의 명제를 얻을 수 있다.

- $a_1 \leq 3 \land n < 10 \Rightarrow n = s$
- $a_1 > 3 \land n < 10 \Rightarrow n - 1 = s$

$$\epsilon_1 = \begin{cases} 0 &(a_1 \leq 3)\ 1 &(a_1 > 3)\ \end{cases} \tag{1} $$

을 정의하면 위 명제들을 명제

$$ n < 10 \Rightarrow n - \epsilon_1 = s \tag{2} $$

로 추상화시킬 수 있다.

n	s	n	s	n	s	n	s	n	s	n	s
13	12	15	13	23	21	25	22	33	30	35	31

n	s	n	s	n	s	n	s	n	s	n	s
39	35	50	36	53	39	55	40	63	48	65	49

ϵ_1 를 사용하여 위 표에서 나타난 자연수 증가 패턴을 귀납적으로 추상화시켜 다음의 명제를 얻을 수 있다.

- $a_2 = 1 \land n < 100 \Rightarrow n - \epsilon_1 - 1 = s$
- $a_2 = 2 \land n < 100 \Rightarrow n - \epsilon_1 - 2 = s$
- $a_2 = 3 \land n < 100 \Rightarrow n - \epsilon_1 - 3 = s$
- $a_2 = 5 \land n < 100 \Rightarrow n - \epsilon_1 - 14 = s$
- $a_2 = 6 \land n < 100 \Rightarrow n - \epsilon_1 - 15 = s$

$$\epsilon _2 = \begin{cases} a_2 &(a_2 \leq 3)\ a_2 + 9 &(a_2 > 3)\ \end{cases} \tag{3} $$

를 정의하면 위 명제들을 명제

$$ n < 100 \Rightarrow n - \epsilon _1 - \epsilon _2 = s \tag{4} $$

로 추상화시킬 수 있다. 그러면 이를 통하여 $n = 99 \Rightarrow s = n - \epsilon _1 - \epsilon _2 = 99 - 1 - 18 = 80$ 을 얻는다. 또한 이로써 n 을 99 증가시키면 s 는 80 이 증가된다는 사실도 얻는다. 이것을 기점으로 다음의 표를 작성할 수 있다.

n	s	n	s	n	s	n	s	n	s
99	80	100	81	199	161	200	162	299	242
300	243	399	323	500	324	599	404	600	405

$\epsilon_1, \epsilon _2$ 를 사용하여 위 표에서 나타난 자연수 증가 패턴을 귀납적으로 추상화시켜 다음의 명제를 얻을 수 있다.

- $a_3 = 1 \land n < 1000 \Rightarrow n - \epsilon_1 - \epsilon_2 - 19 = s$
- $a_3 = 2 \land n < 1000 \Rightarrow n - \epsilon_1 - \epsilon_2 - 38 = s$
- $a_3 = 3 \land n < 1000 \Rightarrow n - \epsilon_1 - \epsilon_2 - 57 = s$
- $a_3 = 5 \land n < 1000 \Rightarrow n - \epsilon_1 - \epsilon_2 - 176 = s$
- $a_3 = 6 \land n < 1000 \Rightarrow n - \epsilon_1 - \epsilon_2 - 195 = s$

$$\epsilon _3 = \begin{cases} a_3 \times 19 &(a_3 \leq 3)\a_3 \times 19 + 9 ^{2}&(a_3 > 3)\ \end{cases} \tag{5} $$

를 정의하면 위 명제들을 명제

$$ n < 1000 \Rightarrow n- \epsilon _1- \epsilon _2 - \epsilon _3 = s \tag{6} $$

로 추상화시킬 수 있다. 그러면 이를 통하여 $n = 999 \Rightarrow s = n- \epsilon _1- \epsilon _2 - \epsilon _3 = 999 - 1 - 18 - 252 = 728$ 을 얻는다. 또한 이로써 n 을 999 증가시키면 s 는 728 이 증가된다는 사실도 얻는다. 이것을 기점으로 다음의 표를 작성할 수 있다.

n	**s**	**n**	**s**	**n**	**s**	**n**	**s**	**n**	**s**
999	728	1000	729	1999	1457	2000	1458	2999	2186
3000	2187	3999	2915	5000	2916	5999	3644	6000	3645
6999	4373								

$\epsilon_1, \epsilon _2, \epsilon _3$ 를 사용하여 위 표에서 나타난 자연수 증가 패턴을 귀납적으로 추상화시켜 다음의 명제를 얻을 수 있다.

- $a_4 = 1 \land n < 10000 \Rightarrow n -\epsilon _1- \epsilon _2 - \epsilon _3 - 271 = s$
- $a_4 = 2 \land n < 10000 \Rightarrow n-\epsilon _1- \epsilon _2 - \epsilon _3 - 542 = s$
- $a_4 = 3 \land n < 10000 \Rightarrow n-\epsilon _1- \epsilon _2 - \epsilon _3 - 813 = s$
- $a_4 = 5 \land n < 10000 \Rightarrow n-\epsilon _1- \epsilon _2 - \epsilon _3 - 2084 = s$
- $a_4 = 6 \land n < 10000 \Rightarrow n-\epsilon _1- \epsilon _2 - \epsilon _3 - 2355 = s$

$$\epsilon _4 = \begin{cases} a_4 \times 271 &(a_4 \leq 3)\ a_4 \times 271 + 9 ^{3}&(a_4 > 3)\ \end{cases} \tag{7} $$

를 정의하면 위 명제들을 명제

$$ n < 10000 \Rightarrow n-\epsilon _1- \epsilon _2 - \epsilon _3 - \epsilon _4 = s \tag{8} $$

로 추상화시킬 수 있다.

2차 추상화

먼저 수열 ${\gamma _{n}} : 1, 9, 99, 999, 9999, \dots$ 을

$$ \gamma *n = 9 \times \displaystyle \sum*{i=1}^{n - 1}10 ^{i-1} \tag{9} $$

로 정의한다(단, $\gamma _1 = 1$).

그렇다면 $(1), (3), (5), (7)$ 을 함수 $f(n)$ 과 수열 $\gamma _n$ 과 $k \in \N$ 에 대하여

$$ \epsilon _k = \begin{cases} a_k f(\gamma_k) &(a_k \leq 3)\ a_k f(\gamma_k) + 9 ^{k-1}&(a_k > 3)\ \end{cases} \tag{10} $$

로 귀납적으로 추상화시킬 수 있다. 이때

$$ \delta (a_k, k) = \begin{cases} 0 &(a _{k} \leq 3)\ 9 ^{k-1} & (a _{k} > 3)\ \end{cases} \tag{11} $$

를 정의하면 (10) 을

$$ \epsilon _{k} = a_k f(\gamma _k) + \delta (a_k, k) \tag{12} $$

로 쓸 수 있다.

또한 $(2), (4), (6), (8)$ 을 $k = \left\lceil \log_{10} n \right\rceil$ 인 $k \in \N$ 대하여

$$ n - \sum_{i=1}^{k} \epsilon *i = s \iff f(n)* = \sum{i=1}^{k}\epsilon _i \tag{13} $$

로 귀납적으로 추상화 시킬 수 있다. 그러므로 ϵ_i 를 (12) 로 치환하여 다음을 얻는다(단, $f(1) = 0$).

$$ \therefore \boxed{f(n) = \sum_{i=1}^{k} \bigg (a_i f(\gamma _i) + \delta (a_i, i)\bigg)} \tag{14} $$

이렇게 $f(n)$ 이 존재한다는 것이 귀납적으로 증명되었다.■

!!! note

```
이때 $f(1)$ 이 있으면 $1 \leq n < 10$ 인 $n$ 에 대하여 $f(n)$ 을 계산할 수 있다.

$f(1), f(9)$ 가 있으면 $10 \leq n < 100$ 인 $n$ 에 대하여 $f(n)$ 을 계산할 수 있
다.

$f(1), f(9), f(99)$ 가 있으면 $100 \leq n < 1000$ 인 $n$ 에 대하여 $f(n)$ 을
계산할 수 있다.

그러므로 $f(\gamma _1), f(\gamma _2), \dots, f(\gamma _{k})$ 가 있으면 $10
^{k-1} \leq n < 10 ^{k}$ 인 $n$ 에 대하여 $f(n)$ 을 계산할 수 있다는 사실을 귀납
적으로 추상화시킬 수 있다.

그러므로 구하고자 하는 $n$ 이 $n < 10 ^{k}$ 라면 $f(\gamma _1), f(\gamma _2),
\dots, f(\gamma _{k})$ 이 있으면 된다. 이러한 $f(\gamma _1), f(\gamma _2),
\dots, f(\gamma _{k})$ 같은 것들은 자주 계산될텐데 이러한 $f(n)$ 의 결과들이 또 다
시 계산되지 않도록 이미 계산된 것을 캐싱해두는 것이 효율적이다.
```

P1 코드 구현

이로써 지금까지의 논의를 통하여 n 과 s 의 차이를 구하는 함수 $f(n)$ 을 코드로 구현할 준비가 되었다. 지금까지의 논의를 기반으로 다음과 같이 매우 쉽게 **Python** 코드를 짤 수 있다. 단지 수식 (14) 와 수식 (14) 가 의존하는 수식 $(9), (11)$ 을 코드로 옮긴 것에 불과하다.

```python
import math

def gamma(n):
    if n == 1:
        return 1

    tmp = 0
    for i in range(1, n):
        tmp += 10 ** (i - 1)

    return 9 * tmp
```

```
def delta(ak, k):
    return 0 if ak <= 3 else 9 ** (k - 1)

f_result = {}

def f(n):
    if n in f_result:
        return f_result[n]
    if n == '1':
        return 0

    k = math.ceil(math.log10(int(n)))
    l = len(n)
    result = 0
    for i in range(1, k+1):
        ai = int(n[l - i])
        result += ai * f(repr(gamma(i))) + delta(ai, i)

    f_result[n] = result
    return result
```

이 코드를 기반으로 다음과 같이 $n = 10 ^{100}$ 일 때 s 를 구해본다.

```
i = 10 ** 100
print(i - f(repr(i)))
```

결과는 다음과 같다.

```
$ python p1.py
2656139888758747693387813220357796268292334526533944959745749617390924909013
0218299438469904400 1
```

시간 복잡도

- `gamma` 함수 : $O(n)$
- `delta` 함수 : $O(1)$
- `f` 함수 : $O(\log_{10} n)$????
    - 솔직히 시간복잡도를 정확히 못 구하겠음 ㅜ

성능 평가

hyperfine 으로 $n = 10 ^{100}$ 에 대한 s 를 구하는 성능을 벤치마킹 해보면 다음과 같이 평균적으로 약 0.094 초 정도 걸린다.

```
$ hyperfine 'python p1.py'
Benchmark #1: python p1.py
  Time (mean ± σ):      94.8 ms ±  12.9 ms    [User: 89.3 ms, System: 4.7
ms]
  Range (min … max):    77.7 ms … 132.8 ms    32 runs
```

Python 코드를 **C++** 로 포팅한 것으로 $n = 999999$ 에 대한 s 를 구하는 성능을 벤치마킹해보면 다음과 같이 평균적으로 0.001 초 걸린다.

```
$ hyperfine ./p1
  Time (mean ± σ):       1.4 ms ±   1.5 ms    [User: 1.4 ms, System: 1.0
ms]
  Range (min … max):     0.0 ms …   9.7 ms    629 runs
```

Python 코드로 $n = 999999$ 에 대한 s 를 구하는 성능을 벤치마킹하면 다음과 같이 평균적으로 0.03 초 걸린다.

```
$ hyperfine 'python p1.py'
Benchmark #1: python p1.py
  Time (mean ± σ):      30.3 ms ±   9.6 ms    [User: 24.4 ms, System: 5.2
ms]
  Range (min … max):    19.0 ms …  58.6 ms    93 runs
```

C++ 로 구현한 것이 **Python** 보다 30 배 더 빠르다.

!!! note

```
직관에서 멀어지려고 오히려 이성으로, 기계적으로 풀려고 했었는데 결국 방향을 못정하게 되
네. 그래서 다시 직관으로 돌아오니까 그제서야 방향이 보이네. 하지만 직관에만 의존하니까
기계적으로 푸는 방식에서의 엄밀함이 부족하네. 또 기억력에는 한계가 있어서 길을 잃어버리
기 쉽네.

이성에만 의존하면 길을 못정하고, 직관에만 의존하면 길을 열심히 갈 수는 있는데 길을 잃어
버리네.

그러면 하이브리드로 해야 하네. 둘 중 어느 것 하나에만 의존하는 태도 자체가 좋지 않네.
직관과 이성 둘 다 장단점이 있으니까.

직관은 길의 방향을 잘 정하고, 이성은 길의 엄밀성을 높혀주니까 먼저 직관에게 방향키를 맡
기긴해야겠네. 근데 직관이 폭주하지 않도록 어느정도 길을 간 후에 이성에게 다시 주권을 주
는 식으로 해야겠네. 근데 이성은 길을 못정하니까 다시 직관에게 주권을 줘야하고.

직관은 빠르고, 자유롭고, 틀에 얽매이지 않고, 문제를 모든 관점에서 바라보게 하네. 근데
휘발성이 크고, 그 빠름과 자유로움이 정리되지 않으면 난잡합이 되버리네.
```

```
이성은 정확하고, 엄밀하고, 자동화될 수 있지만 느리고, 방향을 정하지 못하고, 이해력이 떨어지네.

실제로 이성에 의존할 때 논의가 상당히 기계적으로 진행되다 보니까 $1, 19, 271, \dots$ 의 의미를 뭔지 몰라버렸잖아. 근데 직관으로 좀만 생각해보면 그 의미를 바로 알 수 있었네.
```